Unión Europea
Fondo Social Europeo
El FSE invierte en tu futuro

Laravel

Controladores

Pepe Devesa

GENERALITAT VALENCIANA
Conselleria d'Educació, Investigació, Cultura i Esport

GOBIERNO DE ESPAÑA
MINISTERIO DE EDUCACIÓN, CULTURA Y DEPORTE

Continguts

1 Controladores con Blade **3**

2 Controladores de un sólo método (invoke) **3**

3 Controladores de varios métodos **5**

4 Rutas, vistas y controladores **6**

4.1 Aunando todas las rutas de un controlador 7

4.2 Renombrando las rutas 8

5 Inyección de dependencias en Laravel **9**

5.1 Ejemplo Respuesta 9

5.2 Usar la respuesta para hacer redirecciones 10

1 Controladores con Blade

Los controladores permiten estructurar mejor el código de nuestra aplicación.

Su principal utilidad radica en liberar a los archivos de rutas de tener que ocuparse también de gestionar cierta lógica común de las peticiones, como el acceso a los datos, validación de formularios, etc.

Además, a medida que la aplicación crezca, el archivo de rutas puede ser demasiado grande si tiene que almacenar también la lógica de cada ruta, y el tiempo de procesamiento del archivo también crecerá. Lo mejor es dividir esa lógica en controladores.

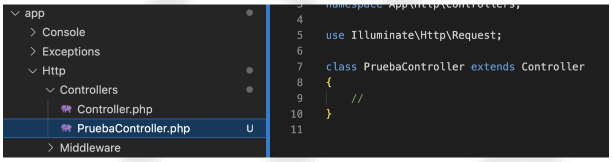
Para definir un controlador en nuestra aplicación, tenemos que echar mano de nuevo del comando php artisan visto previamente. En concreto, utilizaremos la opción *make:controller* seguida del nombre que le queramos dar al controlador. Típicamente, los nombres de controladores terminan con el sufijo `Controller`. Por defecto, los controladores se guardan en la subcarpeta `app/Http/Controllers` de nuestro proyecto Laravel.

2 Controladores de un sólo método (invoke)

Podemos crear uno de prueba así:

```
php artisan make:controller PruebaController
```

Nos creará un controlador vacio:



```
namespace App\Http\Controllers;

use Illuminate\Http\Request;

class PruebaController extends Controller
{
    //
}
```

(a) controlador

Con el parámetro `-i`, que crea el controlador con un método llamado `__invoke`, que se auto ejecuta cuando es llamado desde algún proceso de enrutamiento. Por ejemplo, si creamos el controlador así:

```
php artisan make:controller PruebaController -i
```

Se creará la clase `PruebaController` en la carpeta `app/Http/Controllers`, con un contenido como éste:

```
<?php
```

```
namespace App\Http\Controllers;

use Illuminate\Http\Request;

class PruebaController extends Controller
{
    /**
     * Handle the incoming request.
     */
    public function __invoke(Request $request)
    {
        //
    }
}
```

Dentro del método `__invoke` podemos definir la lógica de generar u obtener los datos que necesita una vista, y renderizarla.

Por ejemplo:

```
<?php

namespace App\Http\Controllers;

use Illuminate\Http\Request;

class PruebaController extends Controller
{
    /**
     * Handle the incoming request.
     */
    public function __invoke(Request $request)
    {
        //
        $libros= array(
            array("titulo"=>"El juego de Ender", "autor"=>"Orson Scott
                Card"),
            array("titulo"=>"La tabla de Flandes", "autor"=>"Arturo Pérez
                Reverte"),
            array("titulo"=>"La historia interminable", "autor"=> "Michael
                 Ende"),
            array("titulo"=>"El Señor de los Anillos", "autor"=>"J.R.R.
                Tolkien")
        );
            return view('listado', compact('libros'));
    }
}
```

Así, en el archivo de rutas, basta con definir la ruta que queramos, y como segundo parámetro del método `get`, indicar el nombre del controlador que se va a disparar para procesar esa ruta. Podemos especificar la clase del controlador, incluyendo con use dicha clase en el archivo de rutas:

```
use App\Http\Controllers\PruebaController;
...
Route::get('prueba', PruebaController::class)->name('prueba');
```

Los controladores de tipo **invoke** son útiles para definir controladores simples, donde sólo necesitamos un método para procesar rutas asociadas a una entidad del programa.

Esto no es lo habitual, ya que, por ejemplo, para una misma entidad (como puedan ser los libros de nuestra biblioteca), necesitaremos métodos para:

- listarlos
- crearlos
- borrarlos, etc.

En este caso, podemos crear otros tipos de controladores que ofrecen más métodos disponibles.

3 Controladores de varios métodos

Si creamos un controlador con la opción `-r` en lugar de la opción `-i` utilizada en el ejemplo anterior, creará un controlador de recursos (resources), y predefinirá en él una serie de métodos de utilidad para las operaciones principales que se pueden realizar sobre una entidad de nuestra aplicación:

- `index` : muestra un listado de los elementos de esa entidad o recurso.
- `create` : muestra el formulario para dar de alta nuevos elementos.
- `store` : almacena en la base de datos el recurso creado con el formulario anterior.
- `show` : muestra los datos de un recurso específico (a partir de su clave o id).
- `edit` : muestra el formulario para editar un recurso existente.
- `update` : actualiza en la base de datos el recurso editado con el formulario anterior.
- `destroy` : elimina un recurso por su identificador.

Obviamente, el código de todos **estos métodos aparecerá vacío** al principio, y los deberemos rellenar con las operaciones correspondientes más adelante.

Vamos a probar esta opción en nuestro proyecto de biblioteca. Crearemos un controlador para gestionar los libros, con este comando:

```
php artisan make:controller -r LibroController
```

De momento varios de los métodos generados en el controlador no los vamos a utilizar. Podemos modificar los dos que sí vamos a usar de momento (`index` y `show`) y poner en ellos lo que antes teníamos en el archivo de rutas. Así nos quedarían, respectivamente:

```
class LibroController extends Controller
{
    /**
     * Display a listing of the resource.
     */
    public function index()
    {
        //
        $libros= array(
            array("titulo"=>"El juego de Ender", "autor"=>"Orson Scott
               Card"),
            array("titulo"=>"La tabla de Flandes", "autor"=>"Arturo Pérez
               Reverte"),
            array("titulo"=>"La historia interminable", "autor"=> "Michael
                Ende"),
            array("titulo"=>"El Señor de los Anillos", "autor"=>"J.R.R.
               Tolkien")
        );
            return view('listado', compact('libros'));
    }
    /**
     * Display the specified resource.
     */
    public function show(string $id)
    {
        //
        return "Mostrando ficha de libro $id";
    }
```

Estas dos rutas quedarían ahora así en el archivo `routes/web.php`

```
use App\Http\Controllers\LibroController;
...
Route::get('libros', [LibroController::class, 'index']);
Route::get('libros/{id}', [LibroController::class, 'show']);
```

4 Rutas, vistas y controladores

A medida que el proyecto crece, generaremos un buen número de vistas asociadas a controladores, y es necesario estructurar estas vistas de una forma adecuada para poderlas identificar rápidamente. Una **convención** que podemos seguir es nombrar las vistas a partir del controlador o modelo al que hacen referencia, y a la operación que realizan. Por ejemplo, si tenemos un controlador llamado

`PruebaController` , se supone que actuará sobre una **tabla llamada pruebas** (lo veremos más adelante, en la sesión de acceso a datos). En nuestro caso de la biblioteca, podemos almacenar las vistas de los libros de la biblioteca en la subcarpeta ‘resources/views/libros“ , y definir dentro las vistas asociadas a cada operación del controlador que tengamos definida. Por ejemplo:

- index.blade.php
- show.blade.php
- …

Paralelamente, cada vez que vayamos a cargar una vista desde algún controlador o ruta, haremos referencia a este nombre. Así, si queremos renderizar la vista `show` para los libros desde el método `show` del controlador de libros, haríamos algo así (pasándole como parámetro el id del libro a buscar, para que lo saque en la vista por ahora):

```
public function show($id) {

   return view('libros.show', compact('id'));

   }
```

Del mismo modo, los nombres que asociemos a las rutas deberían seguir este mismo patrón.

4.1 Aunando todas las rutas de un controlador

Las rutas pueden agruparse en una sola usando el método `resource` de la clase Route , en lugar de get , indicándole como parámetros el nombre base de la ruta, y el controlador que se va a encargar de ella:

```
Route::resource('libros', LibroController::class);
```

IMPORTANTE: a través de la ruta `Route::resource` sólo se incluyen los métodos estándar de un controlador de recursos (`index` , `show` , `create` , `edit` , `etc`), pero NO los que añadamos a mano después en dicho controlador con otros nombres.

También podemos utilizar el método `only` para indicar para qué métodos queremos rutas:

```
Route::resource('libros', LibroController::class)->only(['index', 'show'])
    ;
```

Desde el lado opuesto, tenemos disponible el método `except` para indicar que se generen todas las rutas excepto aquellas para los métodos indicados:

```
Route::resource('libros', LibroController::class)->except(['update', 'edit
    ']);
```

4.2 Renombrando las rutas

Si generamos rutas automáticas para los métodos de un controlador, veremos que para los formularios de crear y editar se define una ruta terminada en /create o en /edit, respectivamente. Esto puede chocar si pretendemos una web hecha en otro idioma. Pero podemos cambiar el nombre que se genera automáticamente para estas rutas, editando el proveedor de servicios `AppServiceProvider`, ubicado en la carpeta *app/Providers*. En el método boot, podemos llamar al método **resourceVerbs** de la clase Route y renombrar los verbos utilizados para acceder a las rutas del recurso. Por ejemplo:

```
public function boot()
{
    Route::resourceVerbs([
        'create' => 'crear',
        'edit' => 'editar'
    ]);
}
```

Deberemos incluir con **use** el espacio de nombres **Illuminate\Support\Facades\Route** para poder emplear la clase Route en el proveedor de servicios.

Para nuestro ejemplo de la biblioteca, podemos devolver un texto plano en los métodos create y edit que indiquen que ahí va un formulario:

```
public function create()
{
    return "Formulario de inserción de libros";
}

public function edit()
{
    return "Formulario de edición de libros";
}
```

Nuestro archivo de rutas se puede quedar con esta única instrucción para todas las rutas de libros, indicando que por ahora sólo vamos a gestionar el listado, la ficha y los dos formularios:

```
Route::resource('libros', LibroController::class)->only(['index', 'show',
    'create', 'edit']);
```

Ahora ya podemos acceder a estas 4 URLs y ver la respuesta correspondiente:

```
http://192.168.1.139:8000//libros (llamará a index)
http://192.168.1.139:8000/libros/3 (llamará a show para el libro 3)
http://192.168.1.139:8000//libros/crear (llamará a create)
http://192.168.1.139:8000/libros/3/editar (llamará a edit para el libro 3)
```

Notar que la URL aparece en castellano gracias a los cambios en AppServiceProvider, pero los nombres de funciones en los controladores siguen estando en inglés, así como en los elementos de la llamada al método only.

Podemos consultar el conjunto de rutas al que está respondiendo nuestra aplicación en todo momento con este comando:

```
php artisan route:list
```

5 Inyección de dependencias en Laravel

El concepto de inyección de dependencias es muy habitual en el uso de frameworks. Consiste en un mecanismo que facilita recursos a los diferentes componentes de la aplicación, y es algo que ya hemos utilizado, sin saberlo, en los métodos que se han generado para los controladores. Por ejemplo, cuando definimos un método en un controlador que necesita procesar una petición, se le pasa como parámetro un objeto de tipo `Request`. Automáticamente, Laravel procesa el tipo de dato y obtiene el objeto asociado (en este caso, la petición del cliente).

```
class PruebaController extends Controller
{
    ...
    public function store(Request $request)
    {
        ...
    }
}
```

5.1 Ejemplo Respuesta

Al igual que tenemos un objeto `Request` para obtener datos de la petición, también existe un `Response` para gestionar la respuesta. Laravel proporciona un método `response` al que le podemos pasar varios parámetros:

- El contenido de la respuesta
- El código de estado HTTP de respuesta (si no se especifica, por defecto es 200)

- Un array con las cabeceras de respuesta (por defecto está vacío).

Así, si por ejemplo queremos emitir una respuesta determinada con su código de estado desde un controlador, podemos hacer esto (por ejemplo, para un código 201):

```
response("Mensaje de respuesta", 201);
```

5.2 Usar la respuesta para hacer redirecciones

Existe también un método `redirect` que podemos emplear para redireccionar a una ruta desde otra, bien especificando la ruta como parámetro…

```
redirect('/');
```

…o bien indicando una ruta con nombre:

```
redirect()->route('inicio');
```

… O si volver a la ruta anterior simplemente podemos usar el método `back`:

```
    return back();
```

Podemos pasar valores a la siguiente redirección, almacenándolos en sesión con el método with, aunque estos valores se perderán en la siguiente petición (no se quedan almacenados en sesión):

```
redirect()->route('inicio')
    ->with('mensaje', 'Mensaje enviado correctamente');
```

Para acceder a este mensaje desde la vista afectada, debemos utilizar la función session:

```
@if(session()->has('mensaje'))
    {{ session('mensaje') }}
@endif
```

Por último, notar que si hacemos la redirección desde dentro de un método de un controlador (por ejemplo, para redigir a una ruta desde otra), deberemos devolver (**`return`**) el resultado de esa redirección para que surta efecto:

```
class LibroController extends Controller
{
    public function index()
    {
        ...
    }

    public function store(...)
    {
        ...
        return redirect()->route('libros.index');
    }
}
```